

Il Modello Netscape²

- Anche alla Netscape si è adottato un modello di tipo *synchronize-and-stabilize*, con opportuni adattamenti allo sviluppo di applicazioni Internet (browser e prodotti server):
 - **dimensione dello staff**
 - in media 1 tester ogni 3 sviluppatori (ma stessa produttività di Microsoft nello sviluppo di prodotti comparabili, ad es. *IE* vs. *Communicator*)
 - **processo**
 - scarso effort di pianificazione (tranne che su prodotti server)
 - documentazione incompleta
 - scarso controllo sullo stato di avanzamento del progetto (lasciato all'esperienza e all'influenza dei project manager)
 - scarso controllo su attività di ispezione del codice (code review)
 - pochi dati storici per il supporto alle decisioni

² M.A. Cusumano and D.B. Yoffie, *Software Development on Internet Time*, *IEEE Computer*, October 1999.

Staffing

Table 2. Allocation of staff in Netscape's client and server development divisions, mid-1998.

	Client products	Server products	Total
Software engineering	110	200	310
Testing (QA)	50	80	130
Product management	50	42	92
Subtotal	210	322	533
Other*	30	98	128
Total	240	420	660

*Persons in activities such as documentation, user interface design, OEM porting, internationalization and localization, and special product support.

Netscape Development Process (1)

Step 1: Product requirements and project proposal

Advance planning meeting (APM) held to brainstorm ideas (marketing, development, executives)

Product vision generated, initially by senior engineers, now mainly by product managers

Some design and coding by engineers to explore alternative technologies or feature ideas

Product requirements document compiled by product managers, with help from developers

Informal review of this preliminary specification by engineers

Functional specification begun by engineers, sometimes with help from product managers

Schedule and budget plan compiled by marketing and engineering, and informally discussed with executives

Step 2: First executive review

Executives review product requirements document and schedule and budget proposal

Plan adjusted as necessary

Netscape Development Process (2)

Step 3: Start of development phase

Design and coding of features, architecture work as necessary

Daily integration of components as they are created and checked in (builds)

Bug lists generated and fixes initiated

Step 4: Interim executive review (if necessary)

Functional specification should be complete at this point

Midcourse corrections in specification or project resources, as necessary

Coordination issues with other products or projects discussed, as necessary

Development continues

Step 5: First internal (alpha) release (takes approximately six weeks)

Development stops temporarily

Intensive debugging and testing of existing code

Alpha release for internal feedback (or possibly a developer's release)

Development continues

User feedback incorporated

Feature-complete target (rarely met, though servers especially try to be as complete as possible)

One week to stabilize beta release

Netscape Development Process (3)

Step 6: Public beta 1 or field test 1 (takes approximately six weeks)

Repeat development and testing steps in Step 5

Server groups moving to “field tests” with limited customers rather than public betas

Step 7: Public beta 2 and 3 (each beta takes approximately six weeks)

Repeat development steps as in Step 5

UI freeze milestone

Feature-complete status “mandatory,” although some minor changes still allowed

Step 8: Code complete

No more code added except to fix bugs; features are functionally complete

Step 9: Final testing and release

Final debugging and stabilization of release candidate

Certification meeting(s) with senior executives for ship decision

Release to manufacturing (RTM) and commercial release

Agile Methods

- In the early 2000's, a reaction was witnessed against the importance of carefully planned software processes, stating that such processes are too restrictive on the developers
- The term *agile method* was introduced to extend the original concept of iterative and incremental development to concepts such as intensive communication within the project, fast feedback, few external rules for the way of working, etc.
- The common values and principles of agile methods are summarized in the *Agile Manifesto*

Agile Manifesto (2001)

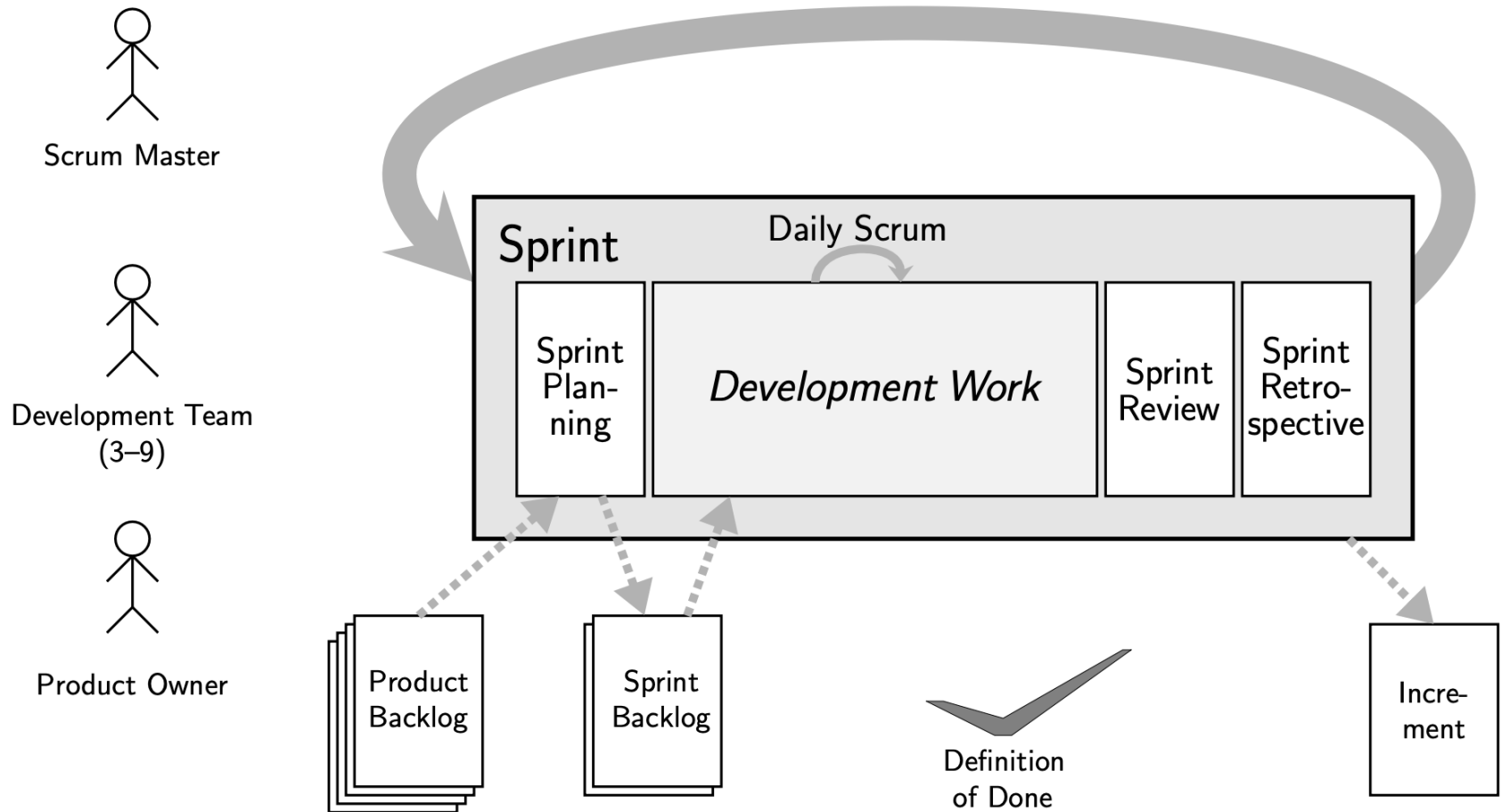
- The Agile Manifesto defines *agile values*

“We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

 - **Individuals and interactions** over *processes and tools*
 - **Working software** over *comprehensive documentation*
 - **Customer collaboration** over *contract negotiation*
 - **Responding to change** over *following a plan*

That is, while there is value in the items on the right, we value the items on the left more.”
- In addition to values, the Agile Manifesto contains the *twelve agile principles* that provide additional guidance on the use of agile methods
- Together, these values and principles define the basic concepts that are today known as *agile development*

Scrum



Scrum Roles

- The *Scrum master*:
 - ensures that the methodology is understood and properly implemented by the development team and the product owner
 - supports the team by helping everyone else to interact with the team according to the Scrum rules
- The *product owner* manages and helps prioritizing the requirements to be implemented as documented in the product backlog
- The *development team* is responsible for developing the product, including all relevant tasks (e.g., designing, coding and testing)

Sprints

- A **sprint** is carried out to deliver a new increment of working software, and typically takes 2 to 4 weeks
- It is started with the **sprint planning** meeting where the Scrum team transfers the items to be developed from the product backlog and transfers them into the sprint backlog
- During the sprint, the development team works on the increment, with short **daily Scrum** meetings (aka stand-up meeting) of the development team to synchronize work and address any problems
- At the end of a sprint, the increment is presented to the product owner and other stakeholders in a **sprint review**
- Finally, the **sprint retrospective** meeting is performed to identify and plan any improvements for the next sprint

Definition of *Done*

- Scrum requires a “definition of done”, where the development team defines for itself what it means for a work item to be done (i.e., before it is allowed to be integrated into the main branch)
- Typical minimum requirements included in this “definition of done” include an adequate number of test cases, as well as checking the integration of the new code to ensure that it does not break the main development branch
- The definition of done also requires that the code has been documented adequately, where the team defines for itself what “adequate” means

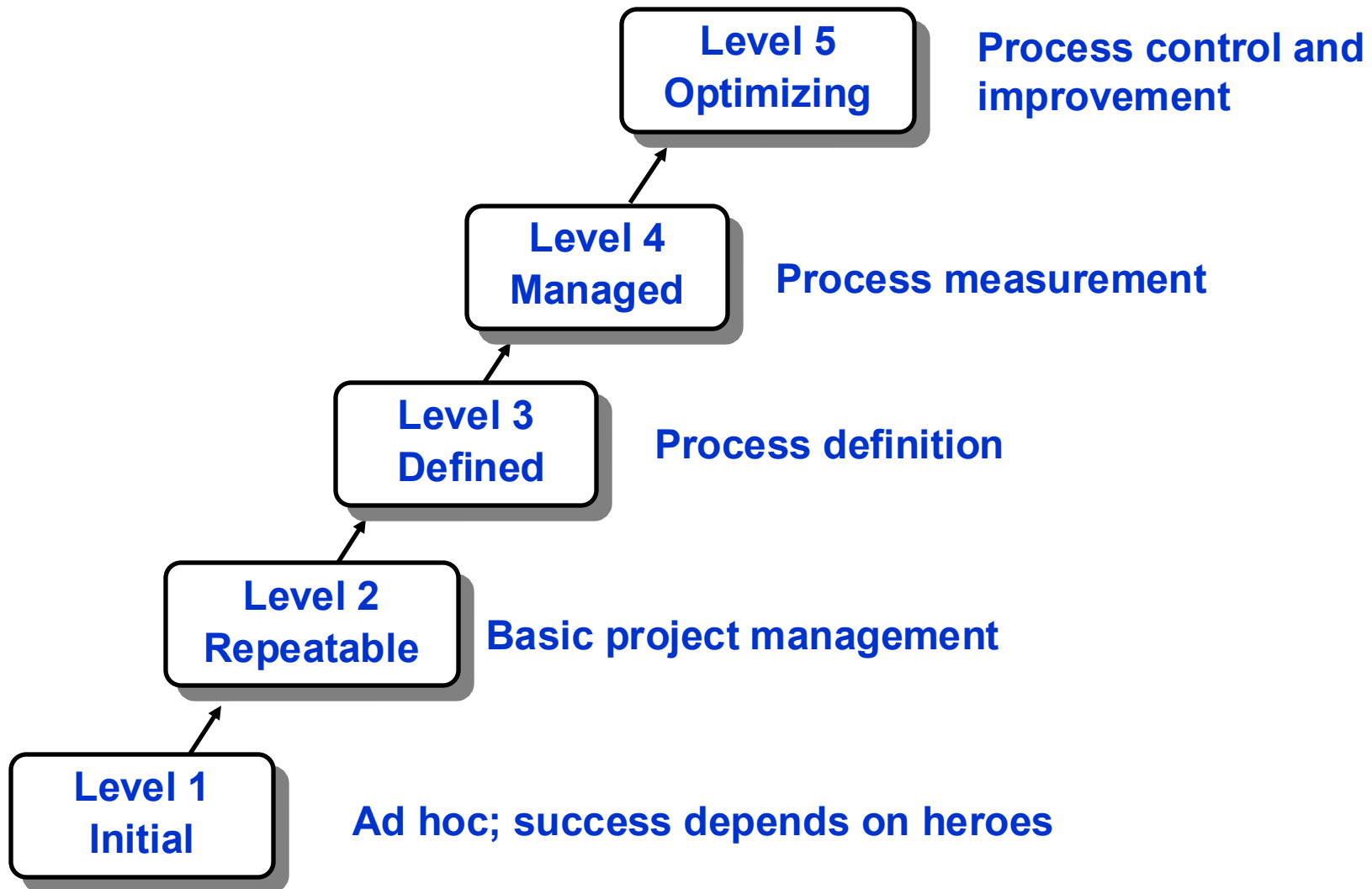
User Stories

- A common practice, used in agile development, often in combination with Scrum (but not defined in the Scrum Guide)
- A **user story**:
 - is a format for describing user requirements as a “story”. It should be short, typically just one sentence, and described from the user point of view
 - uses a common template, such as
 - *As a <role>, I want <goal> so that <benefit>*
 - Example: “As a process engineer, I want to see the dependencies between different process steps so that I can easily verify and validate them”
- Large user stories which are later broken down into smaller ones are often called **epics**

Capability Maturity Model (CMM)

- Il **SEI** (**Software Engineering Institute**) ha predisposto, a partire dal 1993, un modello per determinare il livello di maturità del processo software di un'organizzazione (ovvero una misura dell'efficacia globale dell'applicazione di tecniche di ingegneria del software)
- Il modello è basato su un questionario ed uno schema valutativo a **cinque livelli**
- Ogni livello comprende tutte le caratteristiche definite per il livello precedente

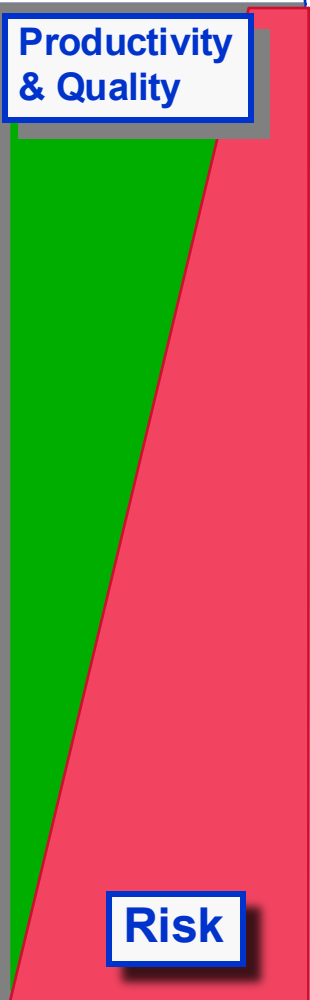
I 5 livelli del CMM



Key Process Areas

- Il CMM associa a ogni livello di maturità alcune **KPA (Key Process Area)**, tra le **18** definite, che descrivono le funzioni che devono essere presenti per garantire l'appartenenza ad un certo livello.
- Ogni KPA è descritta rispetto a:
 - obiettivi
 - impegni e responsabilità da assumere
 - capacità e risorse necessarie per la realizzazione
 - attività da realizzare
 - metodi di "monitoring" della realizzazione
 - metodi di verifica della realizzazione

CMM KPAs

			Result
Level	Characteristic	Key Process Areas	Productivity & Quality
<i>Optimizing</i> (5)	Continuous process capability improvement	Process change management Technology change management Defect prevention	
<i>Managed</i> (4)	Product quality planning; tracking of measured software process	Software quality management Quantitative process management	
<i>Defined</i> (3)	Software process defined and institutionalized to provide product quality control	Peer reviews Intergroup coordination Software product engineering Integrated software management Training program Organization process definition Organization process focus	
<i>Repeatable</i> (2)	Management oversight and tracking project; stable planning and product baselines	Software configuration management Software quality assurance Software subcontract management Software project tracking & oversight Software project planning Requirements management	
<i>Initial</i> (1)	Ad hoc (success depends on heroes)	"People"	Risk

Statistiche a Febbraio 2000

La lista delle organizzazioni a livello 4 e 5 (*maturità elevata*) include:

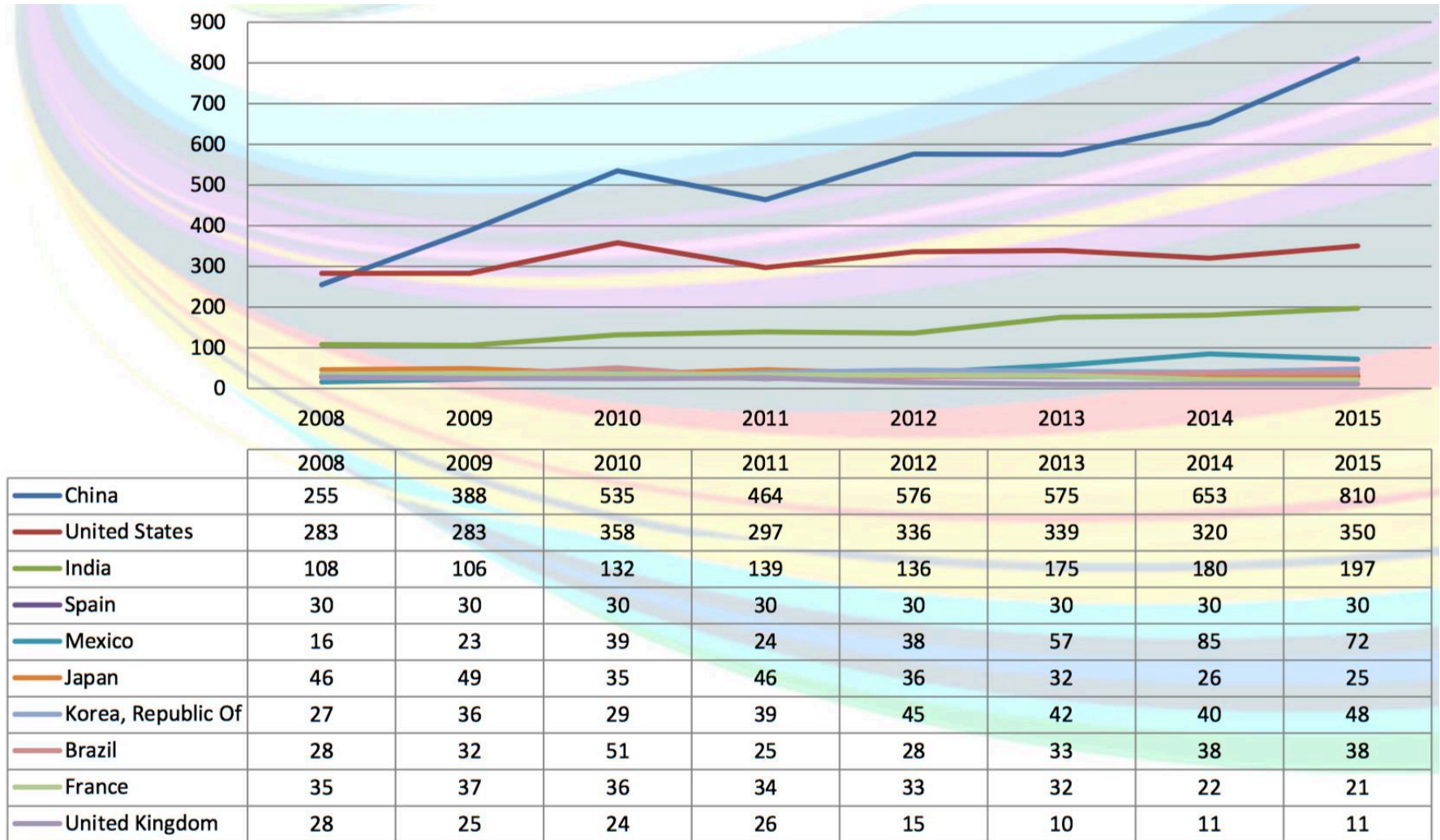
– 71 organizzazioni negli USA

- 44 organizzazioni a Livello 4 (tra cui Oracle, NCR, Siemens Info Systems, IBM Global Services)
- 27 organizzazioni a Livello 5 (tra cui Motorola, Lockheed-Martin, Boeing, Honeywell)

– 25 organizzazioni al di fuori degli USA

- 1 organizzazione a Livello 4 in Australia
- 14 organizzazioni a Livello 4 in India
- 10 organizzazioni a Livello 5 in India

Number of appraisals by country (06/15)



Trends (as of June 2015)

